

# ZAP-WS: Per-Stream Independent Flow Control with Tail-Latency Guarantees and Post-Quantum Confidentiality

ZAP Protocol Authors  
zap-proto.io

2026

## Abstract

We present ZAP-WS, a streaming and publish/subscribe layer over the ZAP transport protocol that provides multiple independent logical streams over a single ZAP connection. Unlike WebSocket (RFC 6455) and HTTP/2 (RFC 9113), which both multiplex streams over a single TCP byte-stream and therefore inherit TCP-level head-of-line (HoL) blocking on packet loss, ZAP-WS gives each stream its own sequence number space, frame-level optional forward error correction (FEC), and per-stream credit-based backpressure. The result is a tail-latency profile in which the arrival latency of stream  $s_i$  is statistically independent of frames in stream  $s_j$  for  $j \neq i$ . ZAP-WS matches the head-of-line blocking elimination of HTTP/3 (RFC 9114) over QUIC, while retaining mutual authentication and post-quantum confidentiality from the underlying ZAP handshake. We formalize the tail-latency claim and prove it under standard packet loss models. We discuss when ZAP-WS is materially better than HTTP/3 (post-quantum deployments, embedded environments without QUIC, scenarios requiring deterministic FEC instead of retransmission).

## 1 Introduction

Modern application traffic is dominated by long-lived streaming patterns: chat, market data, telemetry, live updates, agent-to-agent coordination, log shipping, and progressive AI inference. The transport options historically available to application developers are constrained:

- **Server-Sent Events (SSE)** – unidirectional (server  $\rightarrow$  client), text-only, single stream per HTTP connection.
- **WebSocket (RFC 6455)** [2] – full-duplex, but a single ordered byte stream per TCP connection. Application-level multiplexing requires explicit framing and forfeits any independence guarantee.
- **HTTP/2 (RFC 9113)** [8] – multi-stream multiplexing over a single TCP connection. The streams share TLS and TCP state; a single packet loss blocks delivery of *all* streams until retransmission, even though the streams are logically independent.
- **HTTP/3 (RFC 9114)** [1] over QUIC (RFC 9000) [4] – multi-stream with per-stream loss recovery, eliminating TCP-level HoL blocking. Adoption lags in embedded, kernel-bypass, broker-to-broker, and trusted-network environments. Post-quantum profiles are still being standardized.

ZAP-WS occupies the same design space as HTTP/3 streams but differs in three respects: (i) the underlying ZAP handshake is post-quantum by default (ML-KEM-768 hybrid with X25519, ML-DSA-65 with Ed25519); (ii) the per-stream loss recovery mechanism may use Reed–Solomon FEC rather than retransmission, eliminating one round-trip on the recovery path; (iii) the API is exposed as a `net.Conn`-shaped object per stream, so existing WebSocket, `io.Reader`/`io.Writer`, and SCTP-style code ports mechanically.

**Thesis.** ZAP-WS provides per-stream independent flow control, eliminating the head-of-line blocking that affects WebSocket and HTTP/2 streams sharing a TCP connection, while retaining mutual authentication and post-quantum confidentiality from the underlying ZAP transport.

**Contributions.**

1. A frame format and stream multiplexing scheme that gives each logical stream its own sequence space, window, and (optional) FEC group.
2. A formal tail-latency independence theorem (Theorem 1) and a throughput composition theorem (Theorem 2), proved under a Bernoulli packet-loss model.
3. A practical comparison with WebSocket, HTTP/2, HTTP/3, MPTCP [3], and SCTP [7].
4. A reference implementation sketch and guidance on when ZAP-WS is materially better than HTTP/3.

## 2 Head-of-Line Blocking Analysis

### 2.1 TCP byte-stream model

A TCP connection delivers bytes in order. Internally, TCP buffers out-of-order segments at the receiver and waits for missing segments to be retransmitted before delivering subsequent in-order data to the application. Concretely, if segments  $S_1, S_2, \dots, S_n$  are sent and  $S_k$  is lost, segments  $S_{k+1}, \dots, S_n$  are buffered at the receiver until  $S_k$  is retransmitted and arrives. The application sees a stall of one retransmission timeout (RTO) or fast-retransmit RTT, regardless of whether the bytes in  $S_{k+1}, \dots, S_n$  are part of the same logical unit of work as  $S_k$ .

### 2.2 HTTP/2 over TCP

HTTP/2 multiplexes *streams* – bidirectional sequences of frames – over a single TCP connection [8]. Frames from different streams are interleaved on the wire. A lost segment carrying frames of stream 7 will, by the TCP semantics above, block delivery of any stream’s frames sent in subsequent segments. Stream 9’s frame may already have arrived at the kernel but the application cannot read it.

### 2.3 HTTP/3 over QUIC

QUIC implements stream multiplexing inside the cryptographic transport itself [4]. Each stream has independent flow control and independent loss detection. A lost packet only delays the streams whose frames it actually contained. HoL blocking at the transport layer is eliminated.

### 2.4 ZAP-WS

ZAP-WS adopts the QUIC philosophy – streams are first-class transport objects, not application-layer multiplexes over a byte stream – but implements them on top of ZAP datagrams (which themselves run over UDP or other packet substrates). Each ZAP-WS stream has:

1. An independent 64-bit sequence space.
2. An independent receive window and credit-based backpressure (WINDOW\_UPDATE frames sent per stream, not per connection).
3. An optional FEC group: a  $(k, m)$  Reed–Solomon code applied across  $k$  data frames producing  $m$  parity frames, allowing the receiver to reconstruct up to  $m$  losses without round-trip retransmission.

## 3 ZAP-WS Construction

### 3.1 Frame format

Each ZAP-WS frame on the wire is:

|         |         |         |         |          |         |
|---------|---------|---------|---------|----------|---------|
| +-----+ | +-----+ | +-----+ | +-----+ | +-----+  | +-----+ |
| Type    | StrmID  | StrmSeq | FECGrp  | FECIndex | Payld   |
| 1B      | varint  | varint  | varint? | varint?  | bytes   |
| +-----+ | +-----+ | +-----+ | +-----+ | +-----+  | +-----+ |

Frame types: DATA, WINDOW\_UPDATE, STREAM\_OPEN, STREAM\_CLOSE, FEC\_PARITY, PING, GOAWAY. The FECGrp and FECIndex fields are present only when the connection negotiated FEC for the stream.

### 3.2 Stream lifecycle

A stream is opened by either peer with STREAM\_OPEN. Streams are identified by 62-bit IDs. Bit 0 distinguishes initiator (client/server), bit 1 distinguishes reliable/unreliable. After STREAM\_CLOSE from both sides, the stream ID is retired but never reused on the same connection.

### 3.3 Per-stream flow control

Each receiver advertises an initial window  $W_0$  (typically 256 KiB) and sends WINDOW\_UPDATE frames as the application drains data. A sender that exhausts a stream's window blocks *only that stream*; other streams are unaffected.

### 3.4 Forward error correction

When FEC is negotiated for a stream with parameters  $(k, m)$ , the sender groups  $k$  consecutive DATA frames into a FEC group, computes  $m$  FEC\_PARITY frames using a systematic Reed–Solomon code [6], and transmits the  $k + m$  frames. The receiver can reconstruct the original  $k$  data frames after receiving any  $k$  of the  $k + m$ , tolerating up to  $m$  losses without retransmission.

**Overhead bound.** The bandwidth overhead of  $(k, m)$  FEC is exactly  $m/k$ . This is independent of which frames are lost.

### 3.5 Cryptographic envelope

ZAP-WS frames travel inside ZAP records that are AEAD-encrypted under a session key established by the ZAP handshake [10]. The handshake is hybrid post-quantum: ML-KEM-768 + X25519 for key encapsulation and ML-DSA-65 + Ed25519 for authentication. Confidentiality and authenticity therefore hold under both classical and quantum adversaries (assuming ML-KEM and ML-DSA assumptions). HTTP/3 does not yet mandate post-quantum profiles; ZAP-WS does, by construction.

## 4 Comparison

## 5 Latency Model

Let a connection carry  $N$  streams  $s_1, \dots, s_N$ . Let  $L(s_i, f)$  denote the application-observed arrival latency of frame  $f$  of stream  $s_i$ . Assume an i.i.d. Bernoulli packet-loss channel with loss probability  $p$  and one-way base delay  $D$ .

|                        | WebSocket | HTTP/2  | HTTP/3   | ZAP-WS    |
|------------------------|-----------|---------|----------|-----------|
| Multi-stream           | app-layer | native  | native   | native    |
| TCP HoL blocking       | yes       | yes     | n/a      | n/a       |
| Per-stream window      | no        | yes     | yes      | yes       |
| Loss recovery          | TCP rtx   | TCP rtx | QUIC rtx | rtx + FEC |
| Post-quantum mandatory | no        | no      | no       | yes       |
| Embedded-friendly      | yes       | no      | limited  | yes       |

Table 1: Streaming protocol comparison.

**Proposition 1** (WebSocket / HTTP/2 over TCP – HoL). *For WebSocket and HTTP/2, the latency of stream  $s_i$ ’s frame  $f$  satisfies*

$$L(s_i, f) \geq D + \mathbb{I}[\text{any earlier byte in TCP order was lost}] \cdot R,$$

where  $R$  is the retransmission delay (bounded below by one RTT, often RTO). Hence  $L(s_i, f)$  is dependent on traffic of all other streams  $s_j$ ,  $j \neq i$ .

**Proposition 2** (ZAP-WS / HTTP/3 – no HoL). *For ZAP-WS, the latency of stream  $s_i$ ’s frame  $f$  satisfies*

$$L(s_i, f) \geq D + \mathbb{I}[a \text{ packet carrying part of } f \text{ in } s_i \text{ was lost}] \cdot R',$$

where  $R'$  is the per-stream recovery delay – either an RTT for retransmission, or 0 if FEC parity within the stream’s FEC group is sufficient to reconstruct  $f$ . The indicator depends only on losses affecting  $s_i$ .

These propositions are formalized and proved in `proofs/streaming-tail-latency/proof.tex` as Theorem 1.

## 6 Use Cases

### 6.1 Market data fanout

A market-data publisher serves  $N$  symbols to many subscribers. With WebSocket, a single packet drop in the symbol-A stream stalls symbol-B quotes. With ZAP-WS, each symbol can be its own stream with its own FEC group, sized to the symbol’s update cadence and tolerable staleness. Tail-latency p99.9 on symbol B is independent of symbol-A loss events.

### 6.2 Agent-to-agent task streams

In multi-agent systems, an orchestrator drives many agents in parallel. Each agent’s task plan, tool calls, and partial results are a logical stream. Per-stream independence means a slow agent (long tool execution, local disk wait) cannot block fast agents’ frames.

### 6.3 Telemetry / log shipping

Distinct services shipping logs over a single ZAP connection get independent backpressure. A burst of error-log volume from service X does not delay metric frames from service Y.

### 6.4 Real-time notifications

Notification services often multiplex many subscriptions per user. With ZAP-WS, a stalled subscription does not delay the rest of the user’s subscriptions.

## 7 Implementation

The reference ZAP-WS implementation exposes the following Go API (adapted from the existing `net.Conn` interface):

```
type Conn interface {
    OpenStream(opts StreamOpts) (Stream, error)
    AcceptStream() (Stream, error)
    Close() error
}

type Stream interface {
    net.Conn    // Read, Write, Close, deadlines
    StreamID() uint64
    SetFEC(k, m int) error
}
```

This signature is intentionally compatible with `golang.org/x/net/quic.Stream` shape; existing code that writes to a QUIC stream ports without substantive change. Bindings exist for C, Rust, Python, Java, JS, OCaml, and the other targets in the `zap-proto.io` family.

## 8 When NOT to use ZAP-WS

ZAP-WS is targeted at *many concurrent logical streams*. For pure bulk file transfer where the natural shape is a single ordered byte stream, the ZAP record interface (no per-stream framing overhead) is preferable. For scenarios where a single very large message is sent periodically with no concurrent traffic, WebSocket or HTTP/2 are adequate; the FEC and per-stream overhead of ZAP-WS would add bytes without latency benefit.

## 9 When ZAP-WS is materially better than HTTP/3

1. **Post-quantum mandates.** HTTP/3 standardization is still phasing in PQ key exchange. ZAP-WS is PQ-by-default today.
2. **FEC-driven tail control.** HTTP/3 recovers lost packets via retransmission, costing at least one RTT. ZAP-WS optionally uses Reed–Solomon FEC, recovering losses with zero added round trips at the cost of  $m/k$  bandwidth. For p99.9 latency in lossy networks (satellite, mobile, congested datacenter east-west), this matters.
3. **Embedded / kernel-bypass.** QUIC implementations are large and userspace-coupled; ZAP-WS has small implementations suitable for microcontrollers and DPDK fast paths.
4. **Trusted-network broker meshes.** ZAP-WS provides mutual-auth-by-default with PQ identities, which simplifies broker mesh deployments compared to bolting mTLS onto HTTP/3.

## 10 Related Work

WebSocket [2], HTTP/2 [8], HTTP/3 [1], QUIC [4], Reed–Solomon coding [6], MPTCP [3], SCTP [7], Google QUIC measurements [5]. Head-of-line blocking has been measured empirically by Wang et al. [9] and others.

## 11 Conclusion

ZAP-WS gives applications independent logical streams over a single authenticated, post-quantum-encrypted connection. The independence is formal: tail latency on one stream is statistically independent of traffic on other streams (Theorem 1, proof in companion document). Aggregate throughput composes additively up to link capacity (Theorem 2). The construction is simple, ports existing WebSocket-style code without API friction, and removes the head-of-line blocking that has been the single biggest practical objection to multiplexing many real-time streams over one connection.

## References

- [1] M. Bishop. HTTP/3. RFC 9114, 2022.
- [2] I. Fette and A. Melnikov. The WebSocket protocol. RFC 6455, 2011.
- [3] A. Ford, C. Raiciu, M. Handley, O. Bonaventure, and C. Paasch. TCP extensions for multipath operation with multiple addresses. RFC 8684, 2020.
- [4] J. Iyengar and M. Thomson. QUIC: A UDP-based multiplexed and secure transport. RFC 9000, 2021.
- [5] Adam Langley, Alistair Riddoch, Alyssa Wilk, Antonio Vicente, Charles Krasnic, Dan Zhang, Fan Yang, Fedor Kouranov, Ian Swett, Janardhan Iyengar, Jeff Bailey, Jeremy Dorfman, Jim Roskind, Joanna Kulik, Patrik Westin, Raman Tenneti, Robbie Shade, Ryan Hamilton, Victor Vasiliev, Wan-Teh Chang, and Zhongyi Shi. The QUIC transport protocol: Design and internet-scale deployment. In *Proc. ACM SIGCOMM*, pages 183–196, 2017.
- [6] Irving S. Reed and Gustave Solomon. Polynomial codes over certain finite fields. *Journal of the Society for Industrial and Applied Mathematics*, 8(2):300–304, 1960.
- [7] R. Stewart. Stream control transmission protocol. RFC 4960, 2007.
- [8] M. Thomson and C. Benfield. HTTP/2. RFC 9113, 2022.
- [9] Xiao Sophia Wang, Aruna Balasubramanian, Arvind Krishnamurthy, and David Wetherall. How speedy is SPDY? In *Proc. USENIX NSDI*, pages 387–399, 2014.
- [10] ZAP Protocol Authors. ZAP handshake: Hybrid post-quantum mutual authentication. Specification, zap-proto.io, 2026.